ML Concepts Explained

A repository-specific learning handbook for the three-member team. Teaching examples are distinguished from measured results.

200 synthetic CSV rows; 40 duplicate rows removed; 160 unique examples (80 human-style, 80 AI-style); 20 synthetic problem groups; 96 training, 32 validation, 32 test samples.

Chapter 1 Machine Learning Foundations

Machine learning estimates patterns from examples rather than encoding every decision as a rule. Supervised learning supplies a feature vector x and target y for each example. Here y is a binary style label, and the trained function estimates a score for ai. This is classification; regression instead estimates a numerical target such as running time. Logistic Regression retains its historical name despite solving classification. A feature is an input measurement, whereas a label is the target. A numerical feature such as line count is not itself an authorship fact.

Training fits parameters. Validation supports choices such as model or threshold. Testing estimates performance after those choices are frozen. Overfitting means learning sample-specific regularities; underfitting means failing to capture useful structure. Increasing tree depth can reduce training error while increasing variance. Logistic regularization can reduce variance at the cost of additional bias. Generalization concerns new examples from the intended use distribution. Our synthetic template experiment does not establish it. Leakage occurs when test information influences fitting or selection. Class imbalance changes the usefulness of accuracy and may require weights or appropriate metrics. The cleaned fixture is balanced, but that says nothing about actual prevalence.

Chapter 2 Dataset Preparation

Collect human submissions with consent and retain task, author, source, date, language, and assistance history. Generate matched outputs using recorded models and prompts. This is a future protocol; the current CSV is generated locally from style templates. Keep the source code as text. Labels are strings human and ai; scikit-learn manages the class order, so inference explicitly finds the index of ai. There is no manual LabelEncoder in this repository.

load_dataset removes missing values, boundary whitespace, invalid labels and blank records; it detects conflicting labels for identical code and removes exact duplicates. Removing a row does not repair uncertain provenance. The cleaned sample count is 160, with 80 in each class. Duplicate detection is exact after stripping, not semantic clone detection. Two renamed copies can still share a solution structure.

Stratification preserves label proportions; grouping keeps related records together. Production uses GroupShuffleSplit, not stratified splitting or cross-validation. Twelve groups form training, four validation, and four test. The resulting classes happen to be balanced. Grouped cross-validation would repeat fits across group folds and is future work. A random row split could put solve_a and its renamed solve_b in different partitions, allowing the model to recognize a template instead of a general authorship signal. Even our disjoint IDs do not prevent shared generator templates across partitions.

Chapter 3 Feature Engineering

Feature extraction converts source text into measurements without execution. tokenize.generate_tokens supplies Python tokens, while ast.parse supplies a tree after syntax analysis. Tokens preserve names and literal spellings. An AST removes much concrete punctuation and formatting, retaining constructs such as FunctionDef, If, Call, and Name. A parse tree ordinarily includes more grammar productions; the AST abstracts many of those details. Our model counts AST constructs; it does not consume a full tree embedding.

For each numeric feature the detailed catalog below supplies the definition, intuition, example, implementation expression and limitations. All numeric features are scaled by StandardScaler in the production pipeline. For values 2 and 4, the training mean is 3 and population standard deviation is 1, yielding −1 and +1. Test values use those same training parameters. Feature selection is not separately implemented. TF-IDF caps the vocabulary at 1,200 token/ngram terms. Naming entropy and cyclomatic complexity are not production features: entropy $H=-\Sigma p \log_2 p$ measures uncertainty in a chosen symbol distribution, whereas cyclomatic complexity estimates independent paths in a control-flow graph. Neither should be confused with identifier length or our selected-construct nesting count.

Detailed catalog of the 18 numeric features

comment_to_code_ratio

Definition and intuition: Comment token count divided by nonblank lines that are not comment-only.

Formula: C / max(1,Lcode). Worked example: One full-line and one inline comment with two code lines: 2/2=1.

```
comments / max(1, code_lines)
```

Interpretation and limitations: Describes explicit annotation density. Docstrings are not comments; tiny snippets can produce large ratios.

average_line_length

Definition and intuition: Mean character length of nonblank lines, including indentation.

Formula: Σ len(line) / max(1,Lnonblank). Worked example: Nonblank lengths 4 and 8 give 6.

```
sum(map(len, nonblank)) / max(1, len(nonblank))
```

Interpretation and limitations: Sensitive to formatting, comments and identifier lengths rather than semantic logic.

average_identifier_length

Definition and intuition: Mean NAME token length after excluding Python keywords, counted by occurrence.

Formula: Σ len(identifier) / max(1,nidentifiers). Worked example: Names x and total have lengths 1 and 5: mean 3.

```
sum(map(len, identifiers)) / max(1, len(identifiers))
```

Interpretation and limitations: Repeated variables and library names influence the mean; it is not entropy.

snake_case_ratio

Definition and intuition: Fraction of identifier occurrences matching the lowercase/underscore regex.

Formula: nmatching / max(1,nidentifiers). Worked example: item, input_list and Result give 2/3.

```
bool(re.fullmatch(r"[a-z][a-z0-9]*(?:_[a-z0-9]+)*", name))
```

Interpretation and limitations: Plain lowercase matches too; it does not require an underscore and is not proof of a naming convention.

camel_case_ratio

Definition and intuition: Fraction matching the lower-initial camelCase regex.

Formula: ncamel / max(1,nidentifiers). Worked example: inputList and total give 1/2.

```
bool(re.fullmatch(r"[a-z]+(?:[A-Z][a-z0-9]*)+", name))
```

Interpretation and limitations: The simple regex misses some acronyms and mixed naming styles.

function_count

Definition and intuition: Number of FunctionDef and AsyncFunctionDef nodes.

Formula: Σ 1[node is function definition]. Worked example: Two def statements give 2 even if one is never called.

```
sum(isinstance(n,(ast.FunctionDef,ast.AsyncFunctionDef)) for n in ast.walk(root))
```

Interpretation and limitations: Lambdas are not counted; calls are not definitions.

loop_count

Definition and intuition: Number of For, AsyncFor and While AST nodes.

Formula: Σ 1[node is selected loop]. Worked example: One for and one while give 2.

```
sum(isinstance(n,(ast.For,ast.AsyncFor,ast.While)) for n in ast.walk(root))
```

Interpretation and limitations: Comprehension generators are not counted, so equivalent refactoring changes this feature.

conditional_count

Definition and intuition: Number of If, IfExp and Match nodes.

Formula: Σ 1[node is selected conditional]. Worked example: One if and one conditional expression give 2.

```
sum(isinstance(n,(ast.If,ast.IfExp,ast.Match)) for n in ast.walk(root))
```

Interpretation and limitations: It does not count every boolean operator or individual match case; it is not cyclomatic complexity.

class_count

Definition and intuition: Number of class definitions.

Formula: Σ 1[node is ClassDef]. Worked example: One class containing two methods still gives 1.

```
sum(isinstance(n,ast.ClassDef) for n in ast.walk(root))
```

Interpretation and limitations: The filter-template dataset does not meaningfully cover object-oriented code.

exception_count

Definition and intuition: Combined count of Try, Raise and ExceptHandler nodes.

Formula: nTry+nRaise+nExceptHandler. Worked example: A try/except with one raise has 3 counted nodes.

```
sum(isinstance(n,(ast.Try,ast.Raise,ast.ExceptHandler)) for n in ast.walk(root))
```

Interpretation and limitations: This combines different constructs; it does not count runtime exceptions or TryStar explicitly.

ast_node_count

Definition and intuition: Count of every node emitted by ast.walk.

Formula: number of nodes in the parsed AST. Worked example: x=1 yields Module, Assign, Name, Store and Constant: 5 nodes.

```
sum(1 for _ in ast.walk(root))
```

Interpretation and limitations: Includes context nodes; counts can vary across Python versions and grow with code size.

max_nesting_depth

Definition and intuition: Maximum number of selected function/control nodes along an AST path.

Formula: max selected-construct count on root-to-node paths. Worked example: A function containing for containing if gives 3.

```
depth(root) with increments for selected FunctionDef/If/For/While/With/Try constructs
```

Interpretation and limitations: Not raw AST depth; some constructs such as AsyncFunctionDef and Match do not increment this exact implementation.

code_length

Definition and intuition: Number of source characters.

Formula: len(source). Worked example: x=1 has length 3.

```
len(source)
```

Interpretation and limitations: Whitespace, comments and names all contribute; character count differs from encoded byte count.

line_count

Definition and intuition: Number of lines returned by splitlines.

Formula: len(source.splitlines()). Worked example: x=1 followed by a newline is one line; a further blank line gives two.

```
len(lines)
```

Interpretation and limitations: Trailing-line behavior follows Python splitlines; formatting changes it.

token_count

Definition and intuition: Count of retained Python tokens, excluding comments, layout and end markers.

Formula: nretained tokens. Worked example: x=1 has NAME, OP and NUMBER: 3.

```
len(tokens)
```

Interpretation and limitations: Keywords and punctuation count; partial tokenization may return partial counts internally.

unique_token_ratio

Definition and intuition: Distinct retained token strings divided by total retained tokens.

Formula: |set(tokens)| / max(1,ntokens). Worked example: Tokens x,+,x give 2/3.

```
len(set(tokens)) / max(1,len(tokens))
```

Interpretation and limitations: Treats identical spellings as identical regardless of syntactic role; short samples can be unstable.

blank_line_ratio

Definition and intuition: Fraction of splitlines entries with no nonwhitespace text.

Formula: (L−Lnonblank) / max(1,L). Worked example: Two nonblank lines plus one blank gives 1/3.

```
(len(lines)-len(nonblank)) / max(1,len(lines))
```

Interpretation and limitations: Measures layout, not readability or provenance; boundary stripping differs between training and inference.

docstring_count

Definition and intuition: Count of module, function, async-function and class containers with an AST-recognized docstring.

Formula: Σ 1[ast.get_docstring(container) is not None]. Worked example: A module docstring and function docstring give 2.

```
sum(ast.get_docstring(n) is not None for n in ast.walk(root) if
isinstance(n,(ast.Module,ast.FunctionDef,ast.AsyncFunctionDef,ast.ClassDef)))
```

Interpretation and limitations: These are string expressions, not tokenizer comments. The synthetic AI templates make this an artificially strong label cue.

Chapter 4 Logistic Regression

Let $z=w^Tx+b$. The sigmoid $\sigma(z)=1/(1+e^{(-z)})$ maps a real score to (0,1). The log-odds satisfy $\log(p/(1-p))=z$. A constant-probability contour is a hyperplane in the transformed feature space. This model can respond linearly to TF-IDF and standardized numeric features even though the raw-source-to-feature mapping is nonlinear.

For independent binary labels $y_i \in \{0,1\}$, likelihood is $\Pi\, p_i^{y_i}(1-p_i)^{(1-y_i)}$. Taking negative logarithms gives binary cross-entropy $-\Sigma[y_i \log p_i+(1-y_i)\log(1-p_i)]$. L2 penalizes squared weights and spreads shrinkage; L1 penalizes absolute weights and can produce exact zeros. The production model uses the installed library default L2 regularization with C=1. Balanced weights multiply examples according to inverse class frequencies; on this balanced training split they are equal. The optimization uses max_iter=1500, which is an iteration limit, not a dataset size or number of trees.

Worked example: w=(0.7,−0.3), x=(2,1), b=−0.5 yields z=0.6, p≈0.6457 and log-odds 0.6. At threshold 0.5 the prediction is AI; at 0.7 it is human, with no retraining. If y=1, log loss is $-\ln(0.6457)\approx 0.4375$. A high probability is not a calibrated guarantee: calibration requires checking observed frequencies on separate data. In this repository C=0.01,1,100 are compared in the experimental runner, not retrospectively claimed as production tuning.

Chapter 5 TF IDF and Text Classification

Bag of Words represents token counts without preserving complete order. N-grams add short local sequences; the project uses unigrams and bigrams. The vectorizer vocabulary is learned from training code only. Default lowercasing means camelCase remains one lowercased word rather than being split into subwords. The regex accepts word tokens including numeric strings and underscores; punctuation such as + is excluded from TF-IDF even though Python token statistics count it. Strings and comments can contribute lexical words.

Sublinear TF is $1+\ln c$ for count $c>0$. Smoothed IDF is $1+\ln((1+N)/(1+df))$. Worked example with documents “return value” and “return result”: N=2; IDF(return)=1 and IDF(value)=$1+\ln(3/2)=1.4055$. Before normalization, the first unigram vector is (1,1.4055,0). Its norm is $\sqrt{(1+1.4055^2)}=1.7249$, so its nonzero entries are approximately 0.5797 and 0.8148. Production also includes bigrams, changing the vector dimension and normalization. This example isolates unigrams to show the arithmetic.

Sparse vectors store nonzero entries efficiently. An unseen token has no vocabulary column, so it contributes nothing to that lexical representation. Names learned from a template may be useful within a dataset but fragile after renaming. TF-IDF is computationally simple and interpretable; it lacks semantic equivalence guarantees. A code fragment can preserve its behavior while changing most of its lexical vector.

Chapter 6 Decision Trees and Random Forests

The implemented tree has a root, internal decision nodes and leaves. Each split compares a feature with a threshold. A leaf stores class proportions; predict_proba returns a probability derived from those counts. Gini impurity is $1-\Sigma p_k^2$. With four human and four AI records, Gini=0.5. A split producing children (3 AI,1 human) and (1 AI,3 human) gives impurity 0.375 in each child, so weighted impurity reduction is 0.125. This illustrates a useful split without implying those counts occur in our fitted tree.

Entropy $H=-\Sigma p_k \log_2 p_k$ and information gain are alternative criteria. For a balanced binary node entropy is 1 bit. A pure split reduces it to zero and gains 1 bit. Production uses default Gini, not entropy, information gain ratio or an explicit entropy feature. Depth 12 and minimum leaf size 2 constrain the standalone tree. Cost-complexity pruning via ccp_alpha is not tuned; its default is used.

A Random Forest fits many randomized trees using bootstrap samples and feature subsampling. Averaging their class probabilities reduces dependence on one tree, although correlated biases can remain. The forest uses 160 estimators, min_samples_leaf=2, balanced class weights, n_jobs=-1 and seed 42. It is bagging-style parallel ensemble learning, not sequential boosting. XGBoost, learning-rate tuning, gradient-boosting objectives and CodeBERT were proposed in the synopsis but are not implemented. Therefore this handbook explains the actual tree and forest rather than presenting XGBoost training as completed work.

Chapter 7 Evaluation Metrics

Use AI as the positive class. TP is an AI-labeled sample predicted AI; TN is human-labeled predicted human; FP is human-labeled predicted AI; FN is AI-labeled predicted human. Matrix rows are actual [human,ai] and columns predicted [human,ai]. Accuracy=(TP+TN)/N; precision=TP/(TP+FP); recall=TP/(TP+FN); specificity=TN/(TN+FP); F1=2TP/(2TP+FP+FN). Zero-division cases return zero in the project metrics helper.

Manual example: TP=8, TN=9, FP=3, FN=2 gives N=22, accuracy=17/22=0.7727, precision=8/11=0.7273, recall=8/10=0.8, specificity=9/12=0.75 and F1=16/21=0.7619. This is a teaching example, not measured CodeTrace output. The actual selected-model test matrix is [[16,0],[0,16]]. All four production metrics equal 1.000 on that fixture.

ROC plots true-positive against false-positive rate across thresholds. ROC AUC measures ranking; the experiment runner computes validation AUC, while the dashboard does not. Precision-recall curves emphasize positive-class behavior under imbalance. Neither ranking metric alone validates probability calibration. Calibration asks whether predictions near 0.8 are correct about 80% of the time on representative unseen data. A detector that predicts human for every record could score 95% accuracy when only 5% are AI. Report confusion matrices, per-class errors and prevalence alongside accuracy.

Chapter 8 Explainable AI

Global explanations describe a fitted model across its feature space; local explanations describe a specific prediction. The project displays global importance and raw per-sample feature values, not a complete local attribution decomposition. For Logistic Regression the chart sorts absolute coefficients. Their sign is discarded, so the chart cannot say whether a feature supports AI or human. scikit-learn sorts string classes alphabetically: ai precedes human, so a positive binary coefficient generally supports the second class human, another reason to handle signs explicitly if extending explanations.

Numeric features are standardized and lexical vectors L2-normalized, but coefficient magnitudes still do not establish causality. Correlated counts can share predictive weight. Tree impurity importance favors some feature types and is training-derived. Permutation importance would shuffle one feature and measure score reduction on separate data; it is not implemented. SHAP is also absent. Editing a name changes identifier length, line length, code length, and lexical tokens simultaneously, so a changed probability cannot be attributed to one measurement without further controlled analysis.

Chapter 9 Backend and System Integration

FastAPI defines HTTP routes and serializes JSON. GET retrieves health or metrics; POST /api/analyze accepts a JSON body with code and language. Pydantic enforces types and length bounds. main.py rejects unsupported language and invalid Python syntax before inference. Validation failures return HTTP 422; a missing model returns 503 through the existing RuntimeError path. The frontend renders a readable message for both string details and Pydantic error arrays.

On Analyze, App.tsx sets busy state, clears the previous result, sends fetch with Content-Type application/json, parses the response, and stores the returned result. FastAPI calls analyze in predict.py. The function loads a trusted local joblib dictionary, extracts features, forms a one-row pandas DataFrame, applies the fitted ColumnTransformer and classifier, and finds the ai probability by class name. The response includes prediction, confidence, ai_probability, model, all numeric features and five descriptive summaries. React displays the estimate, probability bar, and signals. No LLM API or source execution participates.

Serialization preserves fitted vocabulary, IDF values, scaler statistics and classifier parameters together. Version metadata and a dataset SHA-256 now accompany the artifact, but there is no signed registry or automatic compatibility migration. Loading occurs per request for simplicity, with an efficiency cost. CORS allows the two local Vite origins. It is a local demonstration deployment rather than a public multi-user service.

Chapter 10 Limitations and Research Considerations

Both labels are synthetic style categories written into a generator, not verified human or LLM provenance. The 20 group IDs vary a few filtering templates and do not represent 20 independent algorithmic tasks. Perfect scores describe template separation only.

Dataset bias includes artificially consistent documentation and naming differences between classes. Domain shift means the deployment input distribution differs from training: factorial, recursion, classes or real assignment solutions may be outside the filter-template fixture. Cross-task evaluation requires independent task families; cross-language evaluation requires separate parsers and data. The current language dropdown only offers Python.

Adversarial refactoring can rename identifiers, change comments and replace loops while preserving behavior. The controlled experiments show sensitivity but do not quantify adversarial success rates across a representative dataset. Human code can be predicted AI and AI code human. False positives matter greatly in education, but this fixture cannot estimate their real prevalence. Ground truth in real mixed-assistance code may be ambiguous. A research-ready collection should record the extent of assistance, consent, model version, prompts, editing history, authors and tasks, with an appropriate split protocol. The system cannot establish academic misconduct, which requires policy context and independent evidence.

Chapter 11 Code Walkthrough

Read the block-by-block source notes below alongside the actual files. The intended order is make_demo_data.py → train.py → features.py → main.py → predict.py → frontend/src/App.tsx. The trainer is an offline process. API inference reads a saved pipeline and never reruns training. Changing feature definitions requires retraining because the saved scaler and learned weights correspond to the old measurements. The model schema was advanced to 2 when comment-ratio semantics were corrected. The experiment runner uses independent experimental fits and leaves the deployed artifact intact.

backend/make_demo_data.py

```
"""Create a labeled synthetic dataset for demonstrating the complete workflow."""
import csv
import random
from pathlib import Path


ROOT = Path(__file__).resolve().parents[1]
HUMAN = [
    "def solve(values):\n    out = []\n    for value in values:\n        if value % 2 == 0:\n
out.append(value)\n    return out\n",
    "def solve(values):\n    # keep the even values\n    result=[]\n    for n in values:\n        if n%2==0:
result += [n]\n    return result\n",
    "def solve(values):\n    evens = []\n    i = 0\n    while i < len(values):\n        current = values[i]\n
if current % 2 == 0:\n            evens.append(current)\n        i += 1\n    return evens\n",
    "def solve(values):\n    picked = []\n    for item in values:\n        even = item % 2 == 0\n        if
even:\n            picked.append(item)\n    return picked\n",
]
AI = [
    '"""Return all even integers from the input sequence."""\ndef solve(values):\n    """Filter values according
to their parity."""\n    return [value for value in values if value % 2 == 0]\n',
    "def solve(values):\n    \"\"\"Select every element divisible by two.\"\"\"\n    filtered_values = [number
for number in values if number % 2 == 0]\n    return filtered_values\n",
    "def solve(values):\n    \"\"\"Build and return a list containing even values.\"\"\"\n    even_values = []\n
for value in values:\n        if value % 2 == 0:\n            even_values.append(value)\n    return
even_values\n",
    "def solve(values):\n    \"\"\"Return values whose remainder after division by two is zero.\"\"\"\n
return list(filter(lambda item: item % 2 == 0, values))\n",
]

def build():
    random.seed(42)
    output = []
    for problem in range(1, 21):
        for label, templates in (("human", HUMAN), ("ai", AI)):
            for variant in range(5):
                code = templates[(problem + variant) % len(templates)]
                code = code.replace("% 2", f"% {problem + 1}")
                code = code.replace("def solve(", f"def solve_{problem}(").replace("def process(", f"def
process_{problem}(")
                # Harmless, deterministic naming variation keeps samples distinct.
                if variant == 1:
                    code = code.replace("values", "items")
                elif variant == 2:
                    code = code.replace("solve", "process")
                elif variant == 3:
                    code = code.replace("return ", "return ") + "\n"
                output.append({"code": code, "label": label, "problem_id": f"problem-{problem:02d}"})
    random.shuffle(output)
    target = ROOT / "data" / "samples.csv"
    with target.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["code", "label", "problem_id"])
        writer.writeheader()
        writer.writerows(output)
    print(f"Wrote {len(output)} synthetic examples to {target}")

if __name__ == "__main__":
    build()
```

backend/features.py

```
"""Static Python code features; submitted source is never executed."""
import ast
import io
import keyword
import re
import tokenize

STRUCTURAL_FEATURES = [
    "comment_to_code_ratio", "average_line_length", "average_identifier_length",
    "snake_case_ratio", "camel_case_ratio", "function_count", "loop_count",
    "conditional_count", "class_count", "exception_count", "ast_node_count",
    "max_nesting_depth", "code_length", "line_count", "token_count",
    "unique_token_ratio", "blank_line_ratio", "docstring_count",
]



def extract_features(source: str) -> dict:
    lines = source.splitlines()
    nonblank = [line for line in lines if line.strip()]
    code_lines = sum(bool(line.strip()) and not line.lstrip().startswith("#") for line in lines)
    identifiers, tokens = [], []
    comments = 0
    try:
        for tok in tokenize.generate_tokens(io.StringIO(source).readline):
            if tok.type == tokenize.COMMENT:
                comments += 1
            elif tok.type == tokenize.NAME and not keyword.iskeyword(tok.string):
                identifiers.append(tok.string)
            if tok.type not in (tokenize.ENCODING, tokenize.ENDMARKER, tokenize.NEWLINE, tokenize.NL,
tokenize.INDENT, tokenize.DEDENT, tokenize.COMMENT):
                tokens.append(tok.string)
    except (tokenize.TokenError, IndentationError):
        pass
    unique = set(tokens)
    id_len = sum(map(len, identifiers)) / max(1, len(identifiers))
    snake = sum(bool(re.fullmatch(r"[a-z][a-z0-9]*(?:_[a-z0-9]+)*", x)) for x in identifiers)
    camel = sum(bool(re.fullmatch(r"[a-z]+(?:[A-Z][a-z0-9]*)+", x)) for x in identifiers)
    counts = {"function_count": 0, "loop_count": 0, "conditional_count": 0, "class_count": 0, "exception_count":
0, "ast_node_count": 0, "max_nesting_depth": 0, "docstring_count": 0}
    try:
        root = ast.parse(source)
        counts["ast_node_count"] = sum(1 for _ in ast.walk(root))
        counts["function_count"] = sum(isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef)) for n in
ast.walk(root))
        counts["loop_count"] = sum(isinstance(n, (ast.For, ast.AsyncFor, ast.While)) for n in ast.walk(root))
        counts["conditional_count"] = sum(isinstance(n, (ast.If, ast.IfExp, ast.Match)) for n in ast.walk(root))
        counts["class_count"] = sum(isinstance(n, ast.ClassDef) for n in ast.walk(root))
        counts["exception_count"] = sum(isinstance(n, (ast.Try, ast.Raise, ast.ExceptHandler)) for n in
ast.walk(root))
        counts["docstring_count"] = sum(ast.get_docstring(n) is not None for n in ast.walk(root) if
isinstance(n, (ast.Module, ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)))
        def depth(node, level=0):
            here = level + isinstance(node, (ast.If, ast.For, ast.AsyncFor, ast.While, ast.With, ast.AsyncWith,
ast.Try, ast.FunctionDef, ast.AsyncFunctionDef))
            return max([here] + [depth(c, here) for c in ast.iter_child_nodes(node)])
        counts["max_nesting_depth"] = depth(root)
    except SyntaxError:
        pass
    return {
        "comment_to_code_ratio": comments / max(1, code_lines),
        "average_line_length": sum(map(len, nonblank)) / max(1, len(nonblank)),
        "average_identifier_length": id_len,
        "snake_case_ratio": snake / max(1, len(identifiers)), "camel_case_ratio": camel / max(1,
len(identifiers)),
        **counts, "code_length": len(source), "line_count": len(lines), "token_count": len(tokens),
        "unique_token_ratio": len(unique) / max(1, len(tokens)),
        "blank_line_ratio": (len(lines) - len(nonblank)) / max(1, len(lines)),
```

```
    }
```

backend/train.py

```
"""Train and evaluate CodeTrace models on a labeled CSV dataset."""
import json
import sys
import hashlib
import platform
import sklearn
from pathlib import Path

import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.model_selection import GroupShuffleSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

try:
    from .features import STRUCTURAL_FEATURES, extract_features
except ImportError:
    from features import STRUCTURAL_FEATURES, extract_features

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data" / "samples.csv"
MODELS = ROOT / "models"
SEED = 42



def load_dataset(path=DATA):
    df = pd.read_csv(path)
    original_rows = len(df)
    required = {"code", "label"}
    if not required.issubset(df.columns):
        raise ValueError("CSV must contain code and label columns.")
    df = df.dropna(subset=["code", "label"]).copy()
    df["code"] = df["code"].astype(str).str.strip()
    df["label"] = df["label"].astype(str).str.strip().str.lower()
    df = df[(df.code != "") & df.label.isin(["human", "ai"])]
    if (df.groupby("code").label.nunique() > 1).any():
        raise ValueError("Identical code has conflicting labels. Resolve these rows before training.")
    df = df.drop_duplicates(subset=["code"])
    if df.label.nunique() != 2 or len(df) < 12:
        raise ValueError("Need at least 12 unique, non-empty examples across human and ai labels.")
    df.attrs["source_rows"] = original_rows
    df.attrs["removed_rows"] = original_rows - len(df)
    fallback_groups = pd.Series([f"sample-{i}" for i in df.index], index=df.index)
    df["group"] = df["problem_id"].fillna(fallback_groups).astype(str) if "problem_id" in df else
fallback_groups
    return df.reset_index(drop=True)



def make_pipeline(model):
    prep = ColumnTransformer([
        ("lexical", TfidfVectorizer(max_features=1200, ngram_range=(1, 2), sublinear_tf=True,
token_pattern=r"(?u)\b\w+\b"), "code"),
        ("structure", StandardScaler(), STRUCTURAL_FEATURES),
    ])
    return Pipeline([("features", prep), ("classifier", model)])
```

```
def metric_row(y, pred):
    return {"accuracy": round(accuracy_score(y, pred), 4), "precision": round(precision_score(y, pred,
pos_label="ai", zero_division=0), 4), "recall": round(recall_score(y, pred, pos_label="ai", zero_division=0),
4), "f1": round(f1_score(y, pred, pos_label="ai", zero_division=0), 4)}



def split_groups(df):
    first = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=SEED)
    trainval_idx, test_idx = next(first.split(df, groups=df.group))
    trainval = df.iloc[trainval_idx]
    second = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=SEED + 1)
    train_idx, val_idx = next(second.split(trainval, groups=trainval.group))
    return df.iloc[trainval.index[train_idx]], df.iloc[trainval.index[val_idx]], df.iloc[test_idx]



def train(data_path=DATA):
    df = load_dataset(data_path)
    source_rows, removed_rows = df.attrs["source_rows"], df.attrs["removed_rows"]
    if df.group.nunique() < 5:
        raise ValueError("Need at least 5 distinct problem_id groups to make train/validation/test splits
without problem leakage.")
    df = pd.concat([df.reset_index(drop=True), pd.DataFrame([extract_features(code) for code in df.code])],
axis=1)
    train_df, val_df, test_df = split_groups(df)
    for name, subset in [("training", train_df), ("validation", val_df), ("test", test_df)]:
        if subset.label.nunique() != 2:
            raise ValueError(f"The {name} split needs both labels. Review problem groups or collect more data.")
    candidates = {
        "Logistic Regression": LogisticRegression(max_iter=1500, class_weight="balanced", random_state=SEED),
        "Decision Tree": DecisionTreeClassifier(max_depth=12, min_samples_leaf=2, class_weight="balanced",
random_state=SEED),
        "Random Forest": RandomForestClassifier(n_estimators=160, min_samples_leaf=2, class_weight="balanced",
random_state=SEED, n_jobs=-1),
    }
    fitted, validation = {}, {}
    for name, estimator in candidates.items():
        pipeline = make_pipeline(estimator)
        pipeline.fit(train_df[["code"] + STRUCTURAL_FEATURES], train_df.label)
        fitted[name] = pipeline
        validation[name] = metric_row(val_df.label, pipeline.predict(val_df[["code"] + STRUCTURAL_FEATURES]))
    selected = max(validation, key=lambda name: (validation[name]["f1"], validation[name]["accuracy"]))
    # Freeze the selected estimator and evaluate once on held-out problems.
    best = fitted[selected]
    test_metrics = {name: metric_row(test_df.label, pipe.predict(test_df[["code"] + STRUCTURAL_FEATURES])) for
name, pipe in fitted.items()}
    predictions = best.predict(test_df[["code"] + STRUCTURAL_FEATURES])
    cm = confusion_matrix(test_df.label, predictions, labels=["human", "ai"]).tolist()
    importance = feature_importance(best)
    MODELS.mkdir(exist_ok=True)
    metadata = {"schema_version": 2, "python": platform.python_version(), "scikit_learn": sklearn.__version__,
"joblib": joblib.__version__, "dataset_sha256": hashlib.sha256(Path(data_path).read_bytes()).hexdigest()}
    joblib.dump({"pipeline": best, "model_name": selected, "features": STRUCTURAL_FEATURES, "metadata":
metadata}, MODELS / "codetrace.joblib")
    result = {
        "artifact_metadata": metadata,
        "selected_model": selected, "selection_metric": "validation F1 (AI as positive class)",
        "validation_metrics": validation, "models": test_metrics, "test_metrics": test_metrics[selected],
        "confusion_matrix": cm, "confusion_matrix_labels": ["human", "ai"], "feature_importance": importance,
        "dataset": {"source_rows": source_rows, "rows": len(df),
"removed_missing_invalid_blank_or_duplicate_rows": removed_rows, "after_validation_and_duplicate_removal":
len(df), "class_distribution": df.label.value_counts().to_dict(), "problem_groups": int(df.group.nunique()),
"split_sizes": {"train": len(train_df), "validation": len(val_df), "test": len(test_df)}, "source":
Path(data_path).name},
        "limitations": "Demonstration data is synthetic and metrics do not indicate real-world detection
performance." if Path(data_path).name == "samples.csv" else "Metrics describe only this dataset and split; they
do not prove authorship or generalize automatically.",
    }
    (MODELS / "metrics.json").write_text(json.dumps(result, indent=2), encoding="utf-8")
    return result
```

```
def feature_importance(pipeline):
    names = pipeline.named_steps["features"].get_feature_names_out()
    clf = pipeline.named_steps["classifier"]
    values = clf.feature_importances_ if hasattr(clf, "feature_importances_") else abs(clf.coef_[0])
    pairs = sorted(zip(names, values), key=lambda p: float(p[1]), reverse=True)[:12]
    return [{"feature": str(name).replace("lexical__", "word: ").replace("structure__", "structure: "),
"importance": round(float(value), 5)} for name, value in pairs]



if __name__ == "__main__":
    result = train(Path(sys.argv[1]) if len(sys.argv) > 1 else DATA)
    print(json.dumps(result, indent=2))
```

backend/predict.py

```
"""Load the saved model and predict from static code features."""
import joblib
import pandas as pd
from pathlib import Path
from .features import STRUCTURAL_FEATURES, extract_features

ARTIFACT = Path(__file__).resolve().parents[1] / "models" / "codetrace.joblib"

def analyze(source: str):
    if not ARTIFACT.exists():
        raise RuntimeError("Model artifact not found. Train the project first with: python backend/train.py")
    artifact = joblib.load(ARTIFACT)
    pipeline = artifact["pipeline"]
    features = extract_features(source)
    row = {"code": source, **features}
    frame = pd.DataFrame([row])
    probabilities = pipeline.predict_proba(frame[["code"] + STRUCTURAL_FEATURES])[0]
    labels = pipeline.classes_
    ai_probability = float(probabilities[list(labels).index("ai")])
    return {
        "prediction": "ai" if ai_probability >= 0.5 else "human",
        "confidence": round(max(ai_probability, 1 - ai_probability), 4),
        "ai_probability": round(ai_probability, 4), "model": artifact["model_name"],
        "features": features,
        "explanation": [
            {"name": "Comment/code ratio", "value": round(features["comment_to_code_ratio"], 3)},
            {"name": "Average identifier length", "value": round(features["average_identifier_length"], 2)},
            {"name": "Functions / loops / conditions", "value": f'{features["function_count"]} /
{features["loop_count"]} / {features["conditional_count"]}'},
            {"name": "AST nodes / nesting depth", "value": f'{features["ast_node_count"]} /
{features["max_nesting_depth"]}'},
            {"name": "Tokens / unique-token ratio", "value": f'{features["token_count"]} /
{features["unique_token_ratio"]:.2f}'},
        ],
    }
```

backend/main.py

```
from pathlib import Path
import json
import ast

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field

from .predict import analyze

ROOT = Path(__file__).resolve().parents[1]
METRICS = ROOT / "models" / "metrics.json"
```

```
app = FastAPI(title="CodeTrace AI", version="1.0.0", description="Static ML-based code style estimate")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
allow_methods=["GET", "POST"], allow_headers=["*"])

class AnalyzeRequest(BaseModel):
    code: str = Field(min_length=20, max_length=20000)
    language: str = "python"

@app.get("/api/health")
def health():
    return {"status": "ok", "model_ready": (ROOT / "models" / "codetrace.joblib").exists()}

@app.get("/api/metrics")
@app.get("/api/models")
def metrics():
    if not METRICS.exists():
        raise HTTPException(503, "Model metrics are unavailable. Train the project first.")
    return json.loads(METRICS.read_text(encoding="utf-8"))

@app.post("/api/analyze")
def analyze_code(request: AnalyzeRequest):
    if request.language.lower() != "python":
        raise HTTPException(422, "Only Python is currently supported.")
    if not request.code.strip():
        raise HTTPException(422, "Paste Python source code to analyze.")
    try:
        ast.parse(request.code)
    except (SyntaxError, ValueError, RecursionError) as exc:
        raise HTTPException(422, "Submit syntactically valid Python source code.") from exc
    try:
        return analyze(request.code)
    except RuntimeError as exc:
        raise HTTPException(503, str(exc)) from exc
```

Logical block walkthrough

make_demo_data.build

Input and output: No arguments; writes data/samples.csv.

Internal logic: Sets seed 42, iterates 20 group numbers and two labels, selects four templates with five variant indices, replaces names and some modulo text, shuffles records and writes CSV.

ML connection and design changes: It creates a software fixture. The template is the true shared unit; problem IDs alone cannot prevent family leakage. Altering labels or templates changes the learning target and requires retraining.

features.extract_features

Input and output: Input source string; output dictionary of 18 numeric features.

Internal logic: Splits lines, tokenizes names/comments/retained tokens, applies naming regexes, parses an AST, counts nodes and traverses selected constructs for depth.

ML connection and design changes: No source executes. Syntax failures are tolerated internally, but public analysis rejects invalid Python. Replacing the ratio or adding features requires refitting the saved transformer and classifier.

train.load_dataset

Input and output: Input CSV path; output cleaned pandas DataFrame with group column and metadata attributes.

Internal logic: Checks columns, drops missing rows, strips code/labels, normalizes labels, filters blank/unknown labels, rejects conflicting duplicates, drops exact duplicates and creates fallback groups.

ML connection and design changes: Cleanup is not provenance verification. Removing duplicates after a random row split would permit leakage; this function runs before splitting.

train.split_groups

Input and output: Input cleaned DataFrame; output train, validation and test DataFrames.

Internal logic: The first seed-42 GroupShuffleSplit reserves 20% of groups for test. A seed-43 split takes 25% of remaining groups for validation.

ML connection and design changes: The resulting row proportions depend on group sizes. Our fixture gives 96/32/32. Groups, not rows, are the splitting unit; no stratification or cross-validation is claimed.

train.make_pipeline

Input and output: Input sklearn classifier; output fitted-ready Pipeline.

Internal logic: ColumnTransformer vectorizes the code column and scales the named numeric columns, then passes the concatenated representation to the classifier.

ML connection and design changes: Keeping transformers in the pipeline prevents a different vocabulary or scale at inference. Calling fit on test data would invalidate evaluation.

train.train

Input and output: Input dataset path; output evaluation dictionary and saved files.

Internal logic: Builds features, creates group partitions, checks both labels, fits three candidates, selects validation F1/accuracy, computes test metrics, saves one pipeline and metrics JSON with metadata.

ML connection and design changes: Capped depth and fixed hyperparameters are design choices, not a search result. The final artifact is training-only fitted. Reusing the test set to choose C would contaminate the reported estimate.

train.metric_row and feature_importance

Input and output: Input labels/predictions or fitted pipeline; output metric or importance dictionaries.

Internal logic: metric_row uses AI-positive precision/recall/F1 with zero_division=0. Importance pairs transformed feature names with absolute coefficients or impurity importances.

ML connection and design changes: Rounding is for reporting. Absolute coefficients discard sign and can be misleading for local interpretation.

predict.analyze

Input and output: Input source string; output JSON-compatible inference dictionary.

Internal logic: Loads joblib, extracts numeric features, creates a DataFrame with code plus numeric columns, computes predict_proba and locates class ai. Applies threshold 0.5 and prepares raw feature summaries.

ML connection and design changes: It never fits or executes source. Per-request loading is simple but less efficient than caching. A metadata schema check could be added later; recorded metadata alone is not enforcement.

main.analyze_code

Input and output: Input validated AnalyzeRequest; output prediction or HTTPException.

Internal logic: Checks Python, rejects whitespace-only source, parses syntax with ast.parse, calls saved inference, maps missing-artifact RuntimeError to status 503.

ML connection and design changes: Pydantic length errors and syntax errors yield 422. Resource limits beyond the character bound are not a complete public-service hardening strategy.

App useEffect and analyze handler

Input and output: Input component state code; output UI state changes and HTTP JSON request.

Internal logic: useEffect fetches metrics once on mount. analyze sets busy, clears old output, POSTs JSON, parses the response, converts string/array error detail to a message, and resets busy in finally.

ML connection and design changes: The browser receives only outputs; the trained model stays in Python. The LOCAL MODEL indicator is static branding rather than a health-monitor signal. Navigation uses state rather than a router.

Experiment runner

Input and output: Input saved artifact and dataset; output CSV, technical inventory and charts.

Internal logic: Runs paired source changes with the deployed model; separately retrains configuration variants on fixed training groups; scores validation only; records split groups and versions.

ML connection and design changes: The factorial probe has unknown provenance. Never describe its score as measured accuracy, and do not force thresholds or code edits until a desired result appears.

Chapter 12 Important Viva Questions

The following questions cover the implementation, mathematical examples, dataset design, integration and research reliability. Answers intentionally challenge the synthetic-data assumptions rather than claiming operational detection capability.

Basic ML concepts

Q1 What does this project actually classify?

Synthetic code-style labels. The generator assigns human and ai strings to templates. Neither label proves actual provenance.

Q2 What is supervised learning?

Learning an input-to-target relationship from labeled examples. Code features are inputs; the CSV label supplies the training target.

Q3 Why is this binary classification?

There are two target classes. The decision threshold maps a probability score to one of human and ai.

Q4 Why is Logistic Regression called regression?

The model regresses log-odds on features. Its output supports binary classification after a sigmoid and threshold.

Q5 What is a feature?

A numerical measurement of the input. For example, comment-token count divided by code-line count describes a source sample.

Q6 What is a label?

The target category assigned to a training sample. It must be supported by collection provenance in real research; ours is a synthetic fixture label.

Q7 What is training?

Fitting transformations and classifier parameters. Vocabulary, IDF, scaler parameters and model coefficients use only training data.

Q8 Why use validation?

To make model choices before test evaluation. The trainer selects maximum validation F1, then accuracy, with a deterministic order tie break.

Q9 What does testing measure?

Performance on an untouched evaluation partition. It estimates only the population represented by that partition and protocol.

Q10 What is overfitting?

Learning patterns that do not generalize. Repeated generator templates can make held-out records easy even with group-disjoint IDs.

Q11 What is underfitting?

A model is too restrictive to capture useful relationships. Strong regularization may flatten probabilities, although current validation classes remain separable.

Q12 Explain bias and variance.

Bias is systematic model error; variance is sensitivity to training samples. Deeper trees can reduce bias while increasing variance; averaging trees can reduce variance.

Q13 What is domain shift?

The input distribution changes between training and use. Real programs or different algorithms need not resemble the filtering templates.

Q14 Why is accuracy alone insufficient?

It hides error types and depends on class prevalence. A constant human predictor can be accurate if AI examples are rare.

Q15 What is data leakage?

Evaluation information affects fitting or selection. Putting near-identical task variants in different partitions can reward memorization.

Q16 Is the dataset balanced?

After cleanup, yes: 80 examples per label. This is an artificial fixture distribution, not measured educational prevalence.

Feature engineering and explanation

Q17 How does tokenization work?

Python tokenize yields lexical tokens. The numeric token statistics include names, operators and literals while excluding comments and layout.

Q18 What is an AST?

A tree of abstract syntax constructs. It captures functions, conditionals and calls while omitting much concrete punctuation.

Q19 How does an AST differ from a parse tree?

It removes grammar details that are not needed for its abstraction. A concrete parse tree tends to retain punctuation and grammar productions; AST nodes represent language constructs.

Q20 What does AST node count include?

All nodes from ast.walk. That includes context nodes such as Load, so it is not merely the number of statements.

Q21 Does nesting depth mean raw AST depth?

No. It increments selected control constructs and function definitions; expression layers alone do not increment it.

Q22 Are comprehensions counted as for loops?

No. The current loop count checks ast.For, ast.AsyncFor and ast.While, not ast.comprehension.

Q23 What is comment-to-code ratio?

Comment-token count divided by nonblank non-comment-only line count. The denominator is bounded below by one; inline comments count, whereas docstrings are string literals.

Q24 Does the snake-case feature require underscores?

No. Its regex also accepts plain lowercase identifiers and some digits after the initial letter.

Q25 Are identifiers counted once each?

No, by occurrence. Repeated uses affect average identifier length and naming-style ratios.

Q26 What is naming entropy?

Entropy of a specified identifier or character distribution. $H=-\Sigma p \log_2 p$; it is not implemented and must not be claimed as a measured feature.

Q27 What is the entropy of two equally likely symbols?

One bit. $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$.

Q28 What is cyclomatic complexity?

A control-flow measure of independent paths. A common graph expression is $E-N+2P$; it is not computed by this project.

Q29 Why scale numerical features?

To put differently sized measurements on comparable standardized scales. The training mean and population variance define later transformations.

Q30 What happens for a constant feature?

StandardScaler handles zero variance safely. It effectively avoids division by zero; a constant centered value contributes no varying numeric signal.

Q31 What is feature selection here?

No separate supervised selector is implemented. The 1,200-term vocabulary cap is a frequency-based vectorizer limit.

Q32 What is global feature importance?

A model-level ranking of measured features. The selected model chart uses absolute coefficients, losing direction.

Q33 Are displayed signals SHAP explanations?

No. They are raw feature summaries; SHAP and permutation importance are not implemented.

Q34 Do positive coefficients mean AI?

Not automatically. Binary scikit-learn coefficients support classes_[1]; with sorted labels this is human.

Q35 Why is feature importance not causal?

Features correlate and training conditions confound them. Renaming changes multiple lexical and numeric properties at once.

Algorithms and mathematics

Q36 What is TF-IDF?

A weighted lexical representation using term frequency and document frequency. Rare terms receive larger IDF; per-document L2 normalization follows weighting.

Q37 What is the actual TF formula?

1+ln(count) for positive counts. sublinear_tf=True dampens repeated-token growth.

Q38 What is the smoothed IDF formula?

1+ln((1+N)/(1+df)). The extra ones avoid zero divisions and follow TfidfVectorizer defaults.

Q39 What are n-grams?

Short adjacent token sequences. The model uses unigram and bigram word features, not full token-order semantics.

Q40 Does TF-IDF retain case?

No, default lowercasing is used. The numeric identifier features still inspect original case and can detect camelCase patterns.

Q41 What happens to an unseen word?

It has no fitted vocabulary column. It adds no lexical contribution, while numeric source features can still change.

Q42 Why are sparse vectors useful?

Most vocabulary coordinates are zero in one short program. Sparse storage avoids representing every absent term explicitly.

Q43 What is the sigmoid of zero?

0.5. It is the central boundary of the standard logistic function.

Q44 What is the sigmoid of 0.6?

Approximately 0.6457. At threshold 0.5 it predicts AI if that probability is defined for the AI class.

Q45 What are log-odds?

ln(p/(1−p)). Logistic Regression models this value as a linear score.

Q46 What is binary cross-entropy?

The negative log-likelihood for binary observations. It penalizes confident wrong probabilities more than uncertain ones.

Q47 What does C control?

Inverse regularization strength. Smaller C shrinks the model more strongly; it is not a probability threshold.

Q48 How do L1 and L2 differ?

L1 penalizes absolute weights; L2 penalizes squared weights. L1 can set coefficients to zero; production uses default L2 behavior.

Q49 What is class_weight balanced?

Inverse-frequency class weighting. For n samples and K classes, class weight is n/(K n_class); the balanced fixture gives equal weights.

Q50 Does max_iter equal epochs or trees?

It is an optimization iteration cap. It does not set the number of data rows or forest estimators.

Q51 How does a decision tree split?

It selects a feature threshold that reduces impurity. The implementation uses Gini by default, with depth and leaf constraints.

Q52 Compute Gini for a balanced node.

0.5. $1-0.5^2-0.5^2=0.5$.

Q53 Compute Gini for a 3:1 node.

0.375. $1-0.75^2-0.25^2=0.375$.

Q54 Is information gain used?

Not the entropy criterion in production. The default Gini reduction drives splits; entropy is explained as an alternative.

Q55 What prevents an excessively complex tree?

max_depth=12 and min_samples_leaf=2. These constrain the standalone tree, although they do not guarantee generalization.

Q56 What is a Random Forest?

An ensemble of randomized decision trees. It averages class probabilities across 160 fitted estimators here.

Q57 What is bootstrap sampling?

Sampling training records with replacement. Different trees see different resampled datasets, increasing diversity.

Q58 Is the forest boosting?

No. Boosting builds models sequentially to improve an objective; this forest averages randomized trees.

Q59 Why did you choose XGBoost?

We did not implement it. It belongs to the synopsis and future scope; the actual project uses three scikit-learn baselines.

Dataset and evaluation

Q60 How many records are used?

160 unique examples from 200 rows. Forty exact duplicates are removed after stripping boundary whitespace.

Q61 How are labels encoded?

They remain strings handled by scikit-learn. Inference finds the ai class index explicitly rather than assuming a fixed column.

Q62 How are missing values handled?

Missing code or label records are removed. Missing problem IDs get per-row fallback groups, weakening protection against shared-task leakage.

Q63 How are conflicting labels handled?

Training now raises a clear ValueError. An identical program should not silently keep whichever contradictory label appears first.

Q64 How are duplicates detected?

Exact code-string equality after stripping. Renamed or reformatted semantic clones remain undetected.

Q65 What is group splitting?

Keeping related records in one partition. GroupShuffleSplit keeps each problem_id exclusively in training, validation or test.

Q66 Do we use stratified splitting?

No. The grouped split happens to be balanced on this fixture; the trainer now checks both labels exist in each partition.

Q67 Do we use cross-validation?

No in production. A stronger study should repeat group-aware validation across independent task families.

Q68 What are the split sizes?

96 train, 32 validation and 32 test. That corresponds to 12, four and four groups for this fixture.

Q69 Are there 20 independent tasks?

No. The IDs reuse a few filter templates with naming and modulus variants.

Q70 Why not use only TF-IDF?

It cannot capture all structural measurements directly. The ablation runner evaluates removing lexical or numeric groups rather than assuming their benefit.

Q71 What is AI precision?

TP/(TP+FP). It is the fraction of AI predictions that match the AI label, not the fraction of all AI samples found.

Q72 What is AI recall?

TP/(TP+FN). It measures the fraction of AI-labeled records predicted AI.

Q73 What is specificity?

TN/(TN+FP). It measures correct rejection of AI among human-labeled records.

Q74 What is F1?

The harmonic mean of precision and recall. It equals 2TP/(2TP+FP+FN) when expressed using confusion counts.

Q75 What does the actual test matrix show?

16 TN and 16 TP, with no FP or FN. The fixture is too small and repetitive to estimate real-world error rates.

Q76 What is ROC AUC?

A threshold-independent ranking summary. It is newly calculated on validation probabilities in the experiments, not the production metrics endpoint.

Q77 What is calibration?

Agreement between predicted probabilities and observed frequencies. An uncalibrated 99% output is not a 99% authorship guarantee.

Q78 Can a threshold change a class without retraining?

Yes. The score stays fixed; only the rule p≥threshold changes.

Q79 Why did threshold metrics stay unchanged?

No validation probabilities crossed the tested boundaries. Unchanged output is a valid observed result, not an experiment failure.

Python implementation

Q80 What does joblib preserve?

The fitted transformer and classifier object graph. Use only trusted artifacts because deserialization is unsafe for untrusted files.

Q81 Does the backend execute source code?

No. It tokenizes and parses text statically; no eval, exec or subprocess runs submitted programs.

Q82 Why are library versions recorded?

Serialized estimators can depend on library behavior. The artifact records schema, Python, scikit-learn, joblib and dataset hash for reproducibility.

Q83 What changes require retraining?

Any change to feature semantics or fitted vocabulary. Old scaler parameters and learned weights assume the previous feature definitions.

Backend and frontend

Q84 What happens on Analyze?

React posts JSON, FastAPI validates, saved inference returns JSON. The fitted transformer and classifier run; training and source execution do not.

Q85 Why use FastAPI?

It exposes typed Python request/response endpoints simply. It connects the Python pipeline to the React client without an LLM API.

Q86 What is Pydantic doing?

Validating request schema and length. The API also explicitly checks supported language and Python syntax.

Q87 What do 422 and 503 mean here?

Invalid input and temporarily unavailable model respectively. They support readable frontend error handling.

Project architecture

Q88 How many models are saved?

Only the selected pipeline. All three metrics are saved, but three independently selectable inference models are not exposed.

Q89 How is language identified?

It is user-selected and validated as Python. There is no automatic language identification model.

Q90 What is the main completed contribution?

An explainable educational pipeline and reproducible sensitivity study. Operational reliability remains unvalidated and must not be claimed during the viva.

Research limitations

Q91 What is feature ablation?

Removing feature groups and retraining. Validation comparisons reveal conditional reliance, though this trivial fixture gives identical class metrics.

Q92 Can renaming change the result?

Yes; the recorded experiment does. The original and renamed programs keep logic but differ in length, naming ratios and vocabulary.

Q93 Does the factorial probe prove generalization?

No. It is one unlabeled unseen task, so accuracy cannot be computed.

Q94 Can it detect an unseen code generator?

There is no evidence for that. No provenance-verified generator data or held-out-model protocol is used.

Q95 What should be improved first?

Collect suitable data and define independent evaluation units. Advanced models cannot repair invalid provenance or template leakage.

Challenging faculty questions

Q96 Why should we trust these results?

They are reproducible software outputs with documented scope. Trust the measurements on this fixture, not a claim of universal authorship detection.

Q97 What if a human writes a docstring?

The model may raise the AI score. Docstrings are a synthetic class cue here; this is an example of why false accusations are possible.

Q98 How is a model tie resolved?

The first candidate wins after equal validation F1 and accuracy. Logistic Regression appears first in the ordered candidate dictionary.

Q99 Is there a real-world accuracy claim?

No. All performance values refer to the repetitive synthetic dataset or explicitly named experiments.